

SGL - Sistema de Gestão de Laboratórios

Relação Completa de Classes, Enums e Componentes

Relação Completa de Classes

Projeto SGL - Sistema de Gestão de Laboratórios

Backend: Java Spring Boot | Frontend: Vue.js | Database: PostgreSQL

Data: 20/07/2026

Resumo Geral

Status	Quantidade	Descrição
' Criado/Funcional	10	Unidade, Laboratório, Usuario, Perfil, DTOs, Repositories, Confin
& p Parcial	2	UsuarioDTO (correções), DataInitializer (adicionar users)
+ Pendente	33	Produto, Pedido, ItemPedido, Projeto, Documento, Movimentacao, Lote + Enums + DTOs + Services +
Total	45	Todos os componentes planejados

1. ENTIDADES (Entities)

1.1. Unidade (Tenant)

' CRIADO

Tabela: unidades

Campos: id (Long, PK) | nome (String) | sigla (String, único) | laboratorios (List<Laboratorio>)

1.2. Laboratório

' CRIADO

1.3. Usuario

' CRIADO

& p OBS: Entidade não tem unidadeId direto (unidade via laboratorio.unidade)

1.4. Produto

+ PENDENTE

Tabela: produtos

Campos: id | laboratorio (FK) | nome | descricao | codigo_referencia | quantidade_atual | quantidade_minima | unidade_medida | localizacao_fisica | ativo | risco: risco (Risco enum) | tipo_risco (TipoRisco enum) | descricao_risco | perecivel (Boolean) | dias_validade | tipo_perecivel (TipoPerecivel enum) | condicoes_armazenamento

1.5. Pedido

+ PENDENTE

1.6. ItemPedido

+ PENDENTE

1.7. Projeto

+ PENDENTE

1.8. Documento

+ PENDENTE

Entidade futura - detalhes a serem definidos

1.9. Movimentacao

+ PENDENTE

Entidade futura - detalhes a serem definidos

1.10. Lote

+ PENDENTE

Entidade futura - detalhes a serem definidos

2. ENUMS

2.1. Perfil

' CRIADO

ADMINISTRADOR | GESTOR | TECNICO | PESQUISADOR | ESTAGIARIO

2.2. StatusPedido

+ PENDENTE

2.3. Risco

+ PENDENTE

2.4. TipoRisco

+ PENDENTE

2.5. TipoPercivel

+ PENDENTE

2.6. TipoMovimentacao

+ PENDENTE

3. DTOs (Data Transfer Objects)

3.1. UnidadeDTO

' CRIADO

Campos: id | nome | sigla

Construtor Entity! DTO: completo

3.2. LaboratorioDTO

' CRIADO

3.3. UsuarioDTO

& p PARCIA

Campos: id | nome | email | senha | perfil | unidadeId | laboratorioId | ativo

CORREÇÕES: Remover unidadeId | Remover senha do construtor | Adicionar perfil/laboratorioId

3.4. ProdutoDTO

+ PENDENTE

3.5. PedidoDTO

+ PENDENTE

3.6. ItemPedidoDTO

+ PENDENTE

3.7. ProjetoDTO

+ PENDENTE

4. REPOSITORIES (Acesso a Dados)

Métodos: findAll, findBySigla

Métodos: findByUnidadeId

Métodos: findByEmail, findByLaboratorioid

Métodos: findByLaboratorioid, findByRisco, findPerciveis, findEstoqueBaixo

Métodos: findByUsuarioid, findByLaboratorioid, findByStatus

Métodos: findByPedidold

Métodos: findByLaboratorioid

Métodos: listarTodos, buscarPorId, salvar, atualizar, deletar

Métodos: listarTodos, listarPorUnidade, buscarPorId, salvar, atualizar, deletar

Métodos: listarTodos, buscarPorId, salvar, atualizar, deletar

Métodos: listarTodos, buscarPorId, salvar, atualizar, deletar, listarEstoqueBaixo, listarPerciveis

Métodos: listarTodos, buscarPorId, criar, aprovar, rejeitar, entregar

Métodos: listarTodos, buscarPorId, salvar, atualizar, deletar

6. CONTROLLERS (Endpoints REST)

6.1. UnidadeController

' CRIADO

GET /api/v1/unidades
POST /api/v1/unidades
GET /api/v1/unidades/{id}
PUT /api/v1/unidades/{id}
DELETE /api/v1/unidades/{id}

6.2. LaboratorioController

' CRIADO

GET /api/v1/laboratorios
POST /api/v1/laboratorios
GET /api/v1/laboratorios/{id}
PUT /api/v1/laboratorios/{id}
GET /api/v1/laboratorios/por-unidade?unidadeId=X

6.3. UsuarioController

+ PENDENTE

GET /api/v1/usuarios
POST /api/v1/usuarios
GET /api/v1/usuarios/{id}
PUT /api/v1/usuarios/{id}
DELETE /api/v1/usuarios/{id}
GET /api/v1/laboratorios/{id}/usuarios

6.4. ProdutoController

+ PENDENTE

GET /api/v1/produtos
POST /api/v1/produtos
GET /api/v1/produtos/{id}
PUT /api/v1/produtos/{id}
DELETE /api/v1/produtos/{id}
GET /api/v1/laboratorios/{id}/produtos
GET /api/v1/produtos/estoque-baixo
GET /api/v1/produtos/risco/{nivel}
GET /api/v1/produtos/pereciveis
GET /api/v1/produtos/validade-proxima

6.5. PedidoController

+ PENDENTE

GET /api/v1/pedidos
POST /api/v1/pedidos
GET /api/v1/pedidos/{id}
PUT /api/v1/pedidos/{id}/aprovar
PUT /api/v1/pedidos/{id}/rejeitar
PUT /api/v1/pedidos/{id}/entregar

6.6. ProjetoController

+ PENDENTE

GET /api/v1/projetos
POST /api/v1/projetos
GET /api/v1/projetos/{id}
PUT /api/v1/projetos/{id}
DELETE /api/v1/projetos/{id}
GET /api/v1/laboratorios/{id}/projetos

7. CONFIGURAÇÃO E INFRAESTRUTURA

7.1. SecurityConfig

' CRIADO

Desabilita CSRF, frames, form login, http basic | Permite /h2-console/**

7.2. CorsConfig

+ PENDENTE



Mapeamento: EntityNotFoundException !' 404

Pendência: Tratar `DataIntegrityViolationException` !' 409

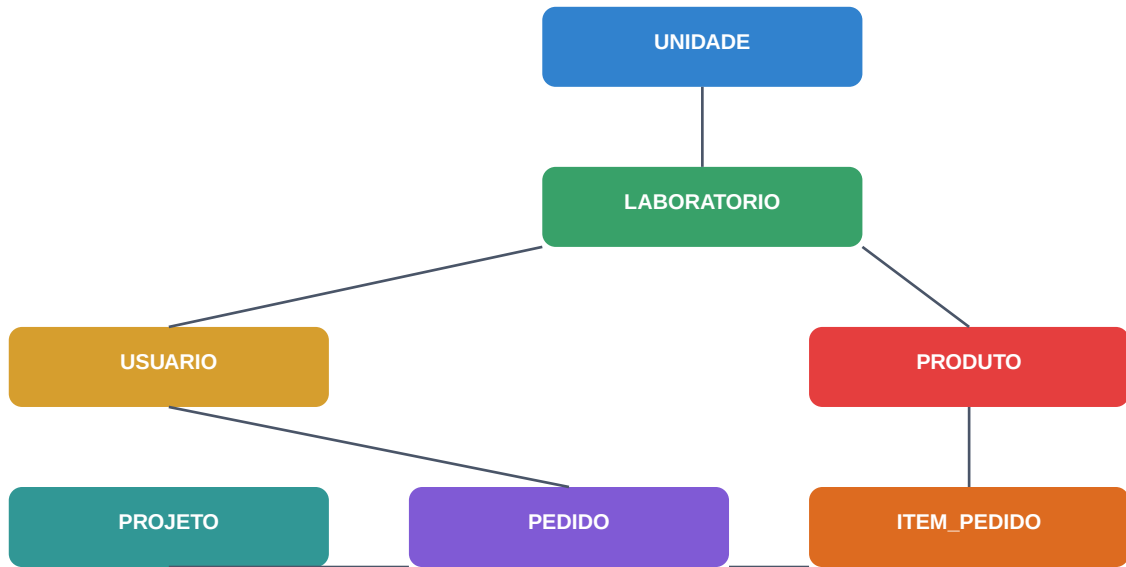
Exception customizada para quando entidade não é encontrada

CommandLineRunner que injeta dados de teste (3 unidades, 5 laboratórios)

Pendência: Adicionar usuários de teste

10. DIAGRAMA DO PROJETO

Diagrama de Classes - Visão Geral



Legenda:

% Unidade	- Tenant/Instituição
% Laboratorio	- Laboratório de pesquisa
% Usuario	- Usuário do sistema
% Produto	- Material em estoque
% Pedido	- Solicitação de material
% ItemPedido	- Item dentro do pedido
% Projeto	- Projeto de pesquisa

Relacionamentos Principais:

- Unidade "1" ! "*" Laboratorio
- Unidade "1" ! "*" Usuario
- Unidade "1" ! "*" Produto
- Laboratorio "1" ! "*" Usuario
- Laboratorio "1" ! "*" Pedido
- Usuario "1" ! "*" Pedido
- Produto "1" ! "*" ItemPedido
- Pedido "1" ! "*" ItemPedido
- Projeto "1" ! "*" Pedido